

Full Stack Development with MERN

Flight Booking and Management System

1. Introduction

1.1 Project Title

Flight Booking and Management System

1.2 Member

Name: Amesh J

Role: Full Stack Developer

Responsibilities

- Built the complete React-based frontend for customers, administrators, and flight operator modules.
 - Designed landing screens, flight search components, booking inputs, and dashboard layouts.
 - Integrated frontend views with backend REST APIs using Axios.
 - Implemented JWT-driven login, authentication checks, and role-based authorization.
 - Modeled MongoDB collections for users, flights, bookings, and operator data.
 - Developed backend APIs in Node.js and Express covering all essential system operations.
 - Implemented features such as seat handling, fare computation, passenger input management, and ticket cancellation.
 - Developed admin dashboards that display statistics for users, bookings, flights, and operator requests.
 - Added operator capabilities for flight creation, updates, and managing bookings tied to their flights.
-

2. Project Overview

2.1 Purpose of the Project

The Flight Booking System is a MERN-based web application created to streamline online flight reservations, ticket management, and operational handling. Customers can search flights, book tickets, and review their booking history. At the same time, Admins and Operators get dedicated dashboards to manage flights, users, bookings, and approval workflows.

The core objective is to offer a secure, scalable, and easy-to-use platform for seamless flight reservation and management.

2.2 Key Features

Customer Features

- Register and log in
- Search flights (one-way or round-trip)
- View available flights by travel date and route
- Book flights with passenger entry forms
- Access booking history
- Cancel confirmed bookings

Admin Features

- View totals: users, operators, flights, and booking metrics
- Approve or decline operator registration requests
- Manage user accounts and bookings
- Monitor and control all flight-related data

Flight Operator Features

- Submit operator approval request
- Add new flight listings
- Edit existing flight details
- View customer bookings for their flights
- Access flight-related statistics on dashboard

System-Level Features

- JWT-secured authentication
 - Role-based access handling (Admin / Customer / Operator)
 - Real-time validations
 - REST API architecture
 - Fully responsive user interface
-

3. Architecture

3.1 Frontend Architecture (React.js)

The frontend is developed using React and follows a structured component-based design.

Key Elements:

- Pages include Landing, Login, Registration, Booking, Admin Dashboard, Operator Dashboard
- Components include cards, search modules, forms, tables, and navigation bars
- Global shared state handled using Context API (authentication and tokens)
- Routing implemented using **react-router-dom**
- Protected routes used for authentication (AuthProtector, LoginProtector)
- Styling implemented with custom CSS and Bootstrap
- Backend API calls handled through Axios

3.2 Backend Architecture (Node.js + Express.js)

The backend exposes RESTful API routes responsible for:

- User registration and login
- Role-based management
- CRUD operations for flights
- Booking creation and modification
- Operator approval workflows

Backend Components:

- JWT authentication middleware
- Role-based access control
- Controllers for modular logic
- Services for shared internal operations

3.3 Database Architecture (MongoDB)

MongoDB collections store primary application data.

User Schema

- Username
- Email
- Hashed password
- User role (customer, admin, operator)
- Operator approval flag

Flight Schema

- Flight identifier and name
- Origin and destination
- Departure & arrival details
- Base price
- Number of seats

Booking Schema

- Associated user ID
- Linked flight ID
- Passenger details
- Journey timing
- Total amount
- Booking status (confirmed / cancelled)

Indexing improves speed during large-scale flight search operations.

4. Setup Instructions

4.1 Prerequisites

Install the following tools:

- Node.js (v16 or above)
- MongoDB (local or Atlas cloud)
- npm or Yarn
- Recommended IDE: Visual Studio Code

4.2 Installation Steps

Step 1 — Clone the repository

```
git clone https://github.com/your-repo-url.git
```

Step 2 — Install Frontend

```
cd client
```

```
npm install
```

Step 3 — Install Backend

```
cd server
```

```
npm install
```

Step 4 — Configure Environment Variables

Create .env inside the server folder:

```
PORT=6001
```

```
MONGO_URL=mongodb://localhost:27017/flightapp
```

```
JWT_SECRET=your_secret_key
```

5. Folder Structure

5.1 Client Folder (React Frontend)

```
client/
```

```
| — src/
```

```
|   | — components/
```

```
| | — pages/
| | — context/
| | — RouteProtectors/
| | — styles/
| | — App.js
| | — index.js
| — public/
```

Explanation

- **components/** — shared reusable UI elements
- **pages/** — all primary screens (Login, Register, Dashboard, Search, etc.)
- **context/GeneralContext.jsx** — manages token, authentication, and global state
- **RouteProtectors/** — ensures access control
- **styles/** — custom CSS

5.2 Server Folder (Backend)

```
server/
| — controllers/
| — models/
| — routes/
| — middleware/
| — server.js
| — .env
```

Explanation

- **controllers/** — handles business logic
- **routes/** — maps API endpoints
- **models/** — MongoDB schemas
- **middleware/** — JWT authentication logic
- **uploads/** — (optional) for storing documents

6. Running the Application

Start Backend

cd server

npm start

Start Frontend

cd client

npm start

URLs

- Frontend → <http://localhost:3000>
 - Backend → <http://localhost:6001>
-

7. API Documentation

Below is a summary of essential API endpoints.

Auth APIs

Method Endpoint Description

POST	/register	Create a new user
POST	/login	Authenticate user and return JWT

Flight APIs

Method	Endpoint	Description
GET	/fetch-flights	Retrieve all flights
GET	/fetch-flight/:id	Get specific flight details
POST	/add-flight	Create a new flight entry
PUT	/update-flight	Edit and update a flight

Booking APIs

Method	Endpoint	Description
POST	/book-ticket	Create a booking
GET	/fetch-bookings	Retrieve user bookings
PUT	/cancel-ticket/:id	Cancel an existing ticket

Admin APIs

Method	Endpoint	Description
GET	/fetch-users	Retrieve all users
POST	/approve-operator	Approve operator account
POST	/reject-operator	Reject operator request

8. Authentication Overview

Authentication involves:

- JWT token creation and verification
- Token stored in localStorage
- Middleware checks for route authorization
- Role-based access for Admin, Customer, and Operator

Security

- Passwords hashed with bcrypt
 - Axios interceptors for automatic token insertion
 - Automatic logout when token becomes invalid
-

9. User Interface Overview

Landing Page

- Contains hero section and platform introduction
- Integrated flight search bar
- Displays results in card-style layout

Login & Register

- Fields for email, password, and user type
- Validation prompts
- Quick links to toggle between pages

Customer Dashboard

- Flight search tools
- List of available flights
- Simple booking interface
- Booking history section

Admin Dashboard

- At-a-glance metrics (users, bookings, flights, operators)
- Access to user, booking, and flight management pages
- Pending operator requests view

Operator Dashboard

- Add flights
- Modify existing flight entries
- View all personal flights
- Check admin approval status

10. Testing

Manual Testing

- Validated forms
- Flight search operations
- Ticket booking and cancellation
- Operator approval workflows
- Authorization checks for restricted pages

Automated Testing (Optional)

- Postman API test collections
 - Basic controller tests using Jest or Mocha
-

11. Demo

A sample demonstration video:

[Flight_booking.mp4](#)

12. Known Issues

- Real-time seat availability not implemented
 - No pagination on large lists
 - Some forms lack detailed validation
 - Email notification service not yet included
-

13. Future Enhancements

- Live seat availability engine
- Graphical seat selection
- Integrated payment gateway
- Email and SMS ticket confirmations
- AI-based support chatbot
- Admin analytics dashboard
- Multi-language support
- Mobile app built with React Native